

Hiranandani Dashboard

How the whole thing works, end to end

React (Vite) + Express · Sperto CRM · no database · generated 23 September 2026

What the app is, what every screen does, what it stores and where, what reaches the client's CRM, and how it is run and deployed. Written against the code as it stands today.

The shape of it

Three screens. One login is one customer. There is no “End Session” button — signing out is what closes a presentation, so it cannot be left half-ended whichever way the staff member leaves.

```
/login          -> /session/start      -> /dashboard
email or        -> Lead ID + device    -> VR tour + projects
Sales ID        (both Sperto-checked) (timed per project)
```

```
Log out from any of them -> session closed, times sent to Sperto
```

Who owns what

- Sperto (the client's CRM) owns the staff list and the customer list. Nothing has to be pre-created here.
- This app owns nothing of its own: no database, no copy of customers, no server-side history.
- What happened during a presentation is logged in the staff member's own browser tab, and the measured times go to Sperto.

The stack, in one line each

Piece	What it is
Web app	React 19 + Vite, plain JSX, react-router-dom. Built to dist/.
API	Express 5, server/app.js, mounted under /api.
Session	Signed httpOnly cookie, verified on every API call.
CRM	Sperto — staff check, lead check, device-usage log.
Store	Process memory only (server/lib/store.js) + the browser tab.
Hosting	Vercel (CDN + one function), or any Node host as one process.

1 · The flow, step by step

1 Sign in — /login

Staff type their email or their Sperto Sales ID (e.g. PDPL0349). No password.
The server asks Sperto whether that identity is real; one they do not have is refused.
Sperto down -> fall back to the built-in roster, so the demo always runs.
Admin / manager have a separate door: email + password, not Sperto-gated.
Reporting is out of scope this release, so admin / manager sign-in returns 403.



2 Who are you, really? — route guard

The browser gets a signed, httpOnly session cookie it cannot read or edit.
Before any screen is drawn the app asks the server to verify it (GET /api/session).
The role comes from that verified token, never from the readable role cookie.
The guard picks the screen; the real enforcement is every endpoint re-checking the cookie.



3 Find the customer — /session/start

The staff member types the Lead ID (or phone). It goes to Sperto through the same endpoint as the login, as type: "lead".
Sperto says no -> the session does not start.
Sperto is down -> fall through to our own list; a customer is waiting.
No Lead ID at all -> "Continue without a Lead ID" makes a walk-in record.
The screen shows the customer nothing about themselves — they are standing there.



4 Pick the device

Tab / TV / Kiosk / Laptop, chosen by the staff member rather than guessed from the browser string — which can only ever say "Chrome · Windows", never "Kiosk".
This is what makes "which device sells the most" answerable.
Sperto is told the session started: an IN call fires here.



5 The presentation — /dashboard

The 360° VR tour plays behind everything. Every project sits in a rail over it.
Tapping one opens that project's own website full-screen, in this tab — not a new browser tab, so the session header, the timers and Log out stay reachable.
A per-project clock starts. Opening another project stops the first one.
Staff can add meeting notes and flag themselves Busy; both are logged.



6 Log out — the only way out

Reachable from the header, and from inside a project's full-screen viewer.
It also happens by itself: 30 minutes idle, an admin force-logout, or the same account signing in elsewhere. All of those run the same code.
The presentation is closed out, and an OUT call carries the times to Sperto.

2 · The parts, and what each one does

The API — every endpoint

Endpoint	Does
POST /api/login	Checks email+password, or Sales ID / email against Sperto. Mints the session cookie.
GET /api/session	Answers who the signed cookie says you are. The route guard's one question.
POST /api/logout	Expires the cookie server-side and drops the leads that sitting claimed.
GET, POST /api/leads	Lead lookup (Sperto), and the claim / status overlay held in memory.
POST /api/session/device-usage	The IN and OUT calls to Sperto, holding the API key server-side.
GET, POST /api/controls	Force-logout, suspend, project blocks — the bits two devices must agree on.
GET, POST /api/inventory	Units-left per project, admin-entered.
GET, POST /api/users	The staff directory and admin-added accounts. Refused to staff.

Guards on every POST

- Same-origin check: a POST with no Origin, or one from another site, is refused (server/lib/csrf.js).
- Rate limiting on login, per client, in memory — per instance, so it is a speed bump, not a wall.
- Authorisation is checked before anything else runs, so a refusal is never reported as a plumbing error.
- Passwords are hashed server-side; the hashes live in a server-only module and never ship to the browser.

The screens

Screen	What it is
/login	One step. Staff door, and the admin / manager door behind a link.
/session/start	Lead lookup, then the device picker. Nothing about the customer shown.
/dashboard	VR tour, project rail, per-project timer, notes, Busy toggle, Log out.
/admin, /manager	The reporting dashboards. Built, but switched off this release.

Two watchers always running

- IdleLogoutWatcher — 30 minutes with no mouse, key, touch or scroll signs out, after a 60-second countdown.
- KickWatcher — polls the controls endpoint, so an admin's force-logout, or the same account signing in elsewhere, lands on this device.

3 · Where everything is stored

There is no database, and that is deliberate. The customer records belong to the client and stay in their CRM. What is left is split between two disposable places.

What	Where	Lives until
Activity log, session data, per-project times	The browser tab's sessionStorage	the tab closes, or sign-out
Lead overlay — who claimed what, status	The API process's memory	that staff member signs out
Staff controls, admin-added accounts, unit counts	The API process's memory	the process restarts

Signing out wipes the first two. The third deliberately survives it — a suspension undone by the very logout it caused would be no suspension at all.

The honest limits

- A restart, a redeploy or a serverless cold start clears the API's memory, and two instances do not share it.
- Nothing in the sales flow breaks when it is empty: an unknown account is simply one nobody has kicked.
- So treat a force-logout as an immediate action rather than a lasting one.
- A manager sees only their own browser's log — it cannot cross devices without a server store, and there isn't one.
- Anyone with devtools can edit the log in their own tab. The append-only guarantee did not survive the move into the browser.

The activity log

Every login, search, customer lookup, project open and close, tab viewed, note written, status toggle and logout, each stamped with the chosen device. The last 5000 events are kept, in this tab only.

login	"Sales Staff signed in"	
step	"Entered presentation"	device=Kiosk
tour_view	"360 VR tour opened"	device=Kiosk
project_open	"Opened Elena"	device=Kiosk
project_close	"Closed Elena"	device=Kiosk (8165ms)
project_open	"Opened Alibaug"	device=Kiosk
project_close	"Closed Alibaug - logged out"	device=Kiosk (14507ms)
step	"Left presentation"	device=Kiosk (39472ms)
logout	"Sales Staff signed out"	

To read it back, in the browser console:

```
JSON.parse(sessionStorage.getItem("hiranandani.activity"))
```

4 · What reaches Sperto

Two integrations, one host: the identity check at login and at lead lookup, and the device-usage log. Both were verified end to end against a stand-in server built to the CRM's own documented quirks.

The rule that governs both

Their server does not use HTTP status codes to mean anything.

```
a rejection arrives as HTTP 200
a success   arrives as HTTP 500      (device-usage, confirmed live)
```

So nothing branches on the status. Every answer is read out of the body, in one place, by both integrations.

This matters most for the device-usage log, which used to discard the response entirely — a visit Sperto refused to store looked exactly like one it stored. It now reports what happened, in Sperto's own words.

Session start — IN

```
POST api_record_device_usage.php
{
  device_id: "3",                <- Kiosk
  lead_id: "WALKIN-1789969341382",
  sales_manager_login: "PDPL0349", <- the Sales ID they signed in with
  type: "IN",
  page_url: ".../session/start"   <- nothing opened yet
}
```

Session end — OUT (this is the one that carries the times)

```
POST api_record_device_usage.php
{
  device_id: "3",
  lead_id: "WALKIN-1789969341382",
  sales_manager_login: "PDPL0349",
  type: "OUT",

  page_url: [                                <- their field, now an array
    { "Elena": 8 },
    { "Alibaug": 15 }
  ],
}
```

page_url is their own field and always has been, so the times travel in it and in nothing else. A custom project_time carrying the same numbers was sent beside it for a while; the client asked for the array alone, and a test now asserts the OUT body carries only their documented fields.

What the answer means

Sperto's reply	We report	What it means
200, status: success	recorded: true	Stored.
500, status: success	recorded: true	Also stored — their host really does this.
200, status: error	recorded: false · rejected	They refused it. Logged with their message.
500, empty body	recorded: false · unavailable	Their known endpoint bug. Never read as “wrong user”.

The route still answers ok: true in every case, on purpose — a presentation must be free to start and to end whether or not the CRM accepted the write. The staff member has already walked away by then.

5 · How time is measured

Per-project time is measured, not inferred from the gap between log entries — that guess is wrong the moment a session ends on a project.

The rules, each covered by a test

Rule	Why it matters
Only projects actually opened appear	A zero in the CRM reads as a visit that happened.
Reopening adds, never duplicates	Elena 3 min + Elena 2 min is one 300s entry, not two.
Two projects never run at once	Opening B banks A's time first, so a second is never counted twice.
A backgrounded tab accrues nothing	A session left open over lunch must not report an hour.
A refresh does not invent time	Time up to the reload is kept; everything after it would be made up.
The project on screen at logout counts	Closing the app is not the same as the customer leaving the room.
Exactly one OUT per presentation	Several exits share one code path; only the first gets to send.
One customer's time never lands on the next	Starting a session clears the slate.

Verified against a real run

A presentation was run in a browser. These are the two records it produced, side by side:

Project	Activity log (measured)	Sent to Sperto	Agree?
Elena	8,165 ms	8 s	yes
Alibaug	14,507 ms	15 s	yes

Alibaug was still open on screen when Log out was pressed — and it was still counted. That is the rule above, working.

6 · Every way a session ends

Several exits, one code path. Before this, only the Log out button reported an OUT — so an idled-out session left Sperto believing the customer was still in the room.

Exit	Who triggers it	Reports OUT
Log out — showcase header	Staff	yes
Log out — inside a project viewer	Staff	yes
Log out — customer lookup screen	Staff	yes
Idle for 30 minutes	By itself, with a 60s warning	yes
Force-logout by an admin	Admin / manager	yes
Same account signs in elsewhere	By itself	yes

What happens, in order

1. close the open project -> logs it, with its measured dwell time
2. close the presentation -> banks the final step's time
3. send OUT to Sperto -> the page_url array, nothing else
4. clear the session cookie -> server-side; the browser cannot do it
5. clear this tab's log -> the next customer starts clean
6. hard navigation to /login

The order is load-bearing. Both log writes are attributed to the active session, so they have to happen before it is cleared — otherwise that project's dwell time and the final step are both silently lost. The navigation happens whatever else fails: clearing local state is best-effort, leaving is not.

7 · Running and deploying it

Locally

```
npm install
cp .env.example .env.local      # then put a SESSION_SECRET in it
npm run dev                     # app on :3000, API on :3001

npm run build                   # -> dist/
npm start                       # dist/ + API from one process, :3001
npm test                        # the Vitest suite
npm run docs:flow               # rebuilds this PDF
```

In dev, Vite proxies /api to the API, so the browser only ever sees one origin — which is what keeps the SameSite cookie and the origin check working. Never split the app and the API across two origins.

On Vercel

```
dist/          served by the CDN
api/index.js -> server/app.js  (all of /api/*)
```

vercel.json carries the build and two rewrites: /api/* to the one function, and everything else to index.html so a hard load of /session/start works. Set SESSION_SECRET before the first deploy — the build succeeds without it and every sign-in then 500s.

Environment

Variable	Needed for
SESSION_SECRET	Required. Signs the session cookie the API verifies.
API_PORT	Optional (default 3001). Where the API listens.
SPERTO_BASE_URL	The CRM that verifies staff and leads, and logs device usage.
SPERTO_API_KEY	Server-side only — never reaches the browser.
SPERTO_DEVICE_USAGE_API_KEY	Separate key for the device-usage log.
SPERTO_TIMEOUT_MS	Optional, default 8000.
VITE_SENTRY_DSN	Optional client-side error reporting. Must be VITE_-prefixed.

Only VITE_-prefixed variables reach the browser. Everything else stays on the server, which is the point — SPERTO_API_KEY in particular must never ship.

Without any credentials at all

- No Sperto keys set: staff sign-in and lead lookup fall back to the built-in accounts and five demo customers.
- Demo accounts: staff@, aditya@, sneha@ (staff123); admin@ (admin123); manager@ (manager123).
- Demo customers: LEAD-1001 ... LEAD-1005, each also findable by phone number.
- Turning Sperto on turns those demo customers off — they are ours, not theirs, so the CRM will refuse them.

8 · What was checked

Every line below was run, not assumed.

Automated

● Unit tests	70 passed · Sperto response reading, per-project time accounting
● Production build	clean
● Lint	0 errors (23 warnings, all pre-existing patterns)
● Secret leak check	no Sperto key anywhere in the built bundle

Live API

Area	Checked
Login	Sales ID accepted / refused, email accepted / refused, wrong password, missing identifier, admin door closed while reporting is off
Cross-site protection	POST with no Origin refused, POST from another origin refused
Session guard	signed in resolves, signed out refused, guard bounces /dashboard to /login
Lead ID	Sperto-known accepted with their name, Sperto-rejected blocks the session, outage falls through
Device usage	IN, OUT with visits, rejection, empty body, bad device type, bad call type, signed out
Admin-only	inventory write, user directory, account creation — all refused for staff

In a real browser

- Signed in by Sales ID, through Sperto.
- A Lead ID Sperto does not have was refused on screen, and the session did not start.
- A walk-in session started with the lead lookup unavailable, using the fallback.
- Two projects opened and closed; the VR tour, the rail, notes and the Busy toggle all rendered and logged.
- Logged out from inside a project's full-screen viewer.
- The resulting Sperto payload and activity log were read back and matched to the second.
- No console errors at any point.

9 · Still open, and not built yet

None of these block the flow. All should be confirmed before the CRM goes live for real staff.

#	Question	Why it matters
1	Does their parser accept <code>page_url</code> as an array of <code>{ project: seconds }</code> ?	It previously received a single URL string. Their server ignores what it does not understand and answers 200 either way — so an old parser looks exactly like success from our side. It is now the only field the times are in.
2	What does a success body actually look like?	We have only ever been shown error bodies. Anything that is not an explicit failure word is read as success.
3	The real <code>device_id</code> per device type	Tab / TV / Kiosk / Laptop currently map to 1 / 2 / 3 / 4 — a placeholder guess.
4	Each staff member's Sperto login code	Without one, that account's sessions skip the device-usage call rather than send a made-up value.
5	Is the login endpoint's 500 fixed?	It was returning HTTP 500 with no body for any type value. Our side survives it either way, but re-test before enabling the key.

Not built yet

- Property data is hardcoded in `src/data/properties.js`; the content API plugs in there.
- Floor plans have no media — a captioned placeholder renders instead.
- Admin and manager reporting dashboards exist as pages but are switched off (`REPORTING_ENABLED`, in both auth modules).

One consequence worth planning for

Turning Sperto on turns the demo customers off.

LEAD-1001 ... LEAD-1005 are ours, not theirs. Once the CRM is answering, it will refuse them - which is correct, and is the whole point of the check. Demos will need a real Lead ID from that moment on.